

Scalable Updatable Secure Storage from Linear-Size Homomorphic Vector Commitments

Long Chen Shuangyu He

Institute of Software, Chinese Academy of Sciences, Beijing 100190

chenlong@iscas.ac.cn, heshuangyu@iscas.ac.cn

Abstract

Updatable secure storage (USS) enables a client to outsource a dynamic encrypted database, maintain a single short local stub, and detect a malicious server that serves stale files—even after current secret keys leak. To allow server-side key rotation without downloading and re-encrypting the data, USS relies on updatable encryption (UE). However, existing USS constructions remain theoretical and inefficient in practice. The only published construction (Chen–Guo–Li–Tang, ASIACRYPT 2023) relies on a Catalano–Fiore homomorphic vector commitment (HVC) with an $O(n^2)$ common reference string (tens of megabytes at $n = 2^{10}$ and prohibitive at $n = 2^{14}$), while leaving the data-encryption mechanism (DEM) unmeasured for large files.

This paper makes USS practical by resolving both bottlenecks. First, we present KHVC, a KZG-based HVC featuring an $O(n)$ SRS, tight q -SDH position binding, and statistical position hiding. At $n = 1024$, parameter generation time drops from 70.2 s to 0.38 s, and operational parameters decrease from 50.5 MB to 96.3 KB. Second, since CGLT23 allows any IND-CPA UE as the DEM (as ciphertext integrity is guaranteed via the stub), we instantiate a measured random-oracle Ring-LWR key-homomorphic PRF sized for 127 correct updates, caching public a_b NTTs to optimize update performance. FileUp openings use a one-hot closed form to eliminate dense FK20 overhead. We implement our scheme and the 2023 baseline in Rust (BLS12-381) and evaluate them on files up to 1 GB. On an Apple M5 Pro, at 1 MB the DEM achieves 32.6 MB/s with a $2\times$ ciphertext expansion and a 4096-byte token.

Keywords: vector commitment, KZG, updatable encryption, updatable secure storage, key rotation.

1 Introduction

Cloud storage services are increasingly widely used for outsourcing massive dynamic databases. To meet standard security policies (e.g., PCI DSS and NIST guidelines) requiring periodic cryptographic key rotation, *Updatable Encryption* (UE) [4, 5, 7, 6] has emerged as a primitive of choice. UE allows a storage server to re-encrypt ciphertexts to a fresh epoch using a light update token generated by the client, without learning the underlying plaintext or requiring the client to download and re-upload the entire dataset.

However, standard UE alone is insufficient for securing dynamic database environments. A malicious or compromised server, even when performing key rotations correctly, can still mount a state-rollback attack by returning stale ciphertexts from earlier epochs that successfully decrypt under the current key. To enforce version integrity and detect such rollback behavior, Chen, Guo, Li, and Tang [12] formalize *Updatable Secure Storage* (USS). In a USS scheme, the client maintains only a single short, constant-size local stub, while the update communication for file and key updates remains independent of the total file count n . Most importantly, USS guarantees authenticity and freshness detection even if current secret keys have been leaked. To achieve this, CGLT23 proposed a modular KEM+DEM blueprint: a Homomorphic Vector

Commitment (HVC) commits to file hashes to form the stub, an updatable KEM (e.g., RISE) protects evaluation openings, and a ciphertext-independent UE acts as the Data Encryption Mechanism (DEM) for payload files.

Motivation & Limitations of Prior Art. Despite its strong theoretical security, existing USS constructions remain far from practical deployment for real-world, large-scale file systems. Specifically, prior work suffers from two major computational and storage bottlenecks:

1. **Explosive SRS/CRS Scale in HVC:** The baseline USS construction [12] relies on the Catalano–Fiore (CF) HVC [2]. Because CF-HVC requires precomputing every pairwise group product $g^{z_i z_j}$, its Common Reference String (CRS) grows quadratically as $O(n^2)$. In practice, for a database of $n = 2^{10}$ files, the parameters exceed 50 MB, and setup time requires over a minute; at $n = 2^{14}$, the parameters quickly balloon to several gigabytes, rendering setup and operation prohibitively expensive.
2. **Unrealized DEM Efficiency for Large Payloads:** While the CGLT23 theoretical framework allows any IND-CPA UE as the DEM (since file integrity is verified via the local stub rather than the DEM itself), prior literature leaves the actual performance of payload encryption unmeasured and unoptimized for large files. Wrapping gigabyte payloads with traditional group-based or curve-based key-homomorphic PRFs introduces unbearable performance degradation and massive ciphertext expansion, while standard symmetric modes like AES-GCM lack native update mechanisms.

1.1 Our Contributions.

In this paper, we bridge the gap between theoretical USS definitions and real-world performance by designing, instantiating, and measuring a fully practical, high-throughput USS system for large-file databases. Our main contributions are summarized as follows:

- **Linear-Size HVC (KHVC):** We introduce KHVC, a succinct KZG-based homomorphic vector commitment with an $O(n)$ Structured Reference String (SRS). KHVC achieves tight q -SDH position binding and statistical position hiding with a succinct auxiliary string $\text{aux} = \rho \in \mathbb{F}_p$. By deriving a closed-form formula for single-slot updates, we eliminate dense FK20 evaluation overhead during FileUp operations. Compared to CF-HVC at $n = 1024$, KHVC reduces parameter size from 50.5 MB to 96.3 KB (over $500\times$ reduction) and setup time from 70.2 s to 0.38 s.
- **High-Throughput Ring-LWR DEM:** Exploiting the insight that USS requires only IND-CPA security for the DEM, we instantiate an efficient lattice-based DEM using a random-oracle Ring-LWR key-homomorphic PRF sized for up to 127 consecutive key updates (sufficient for decades of periodic rotation). By pre-caching public NTT representations and removing redundant header embeddings from the body ciphertexts, we achieve near-native payload update speeds with moderate expansion.
- **Implementation & End-to-End Evaluation:** We implement the complete USS architecture alongside the 2023 baseline in Rust using BLS12-381. On an Apple M5 Pro workstation, our Ring-LWR DEM achieves a re-encryption throughput of 32.6 MB/s on 1 MB payloads with a $2\times$ ciphertext expansion, demonstrating for the first time that USS can scale efficiently to gigabyte-sized cloud databases.

2 Preliminaries

Let $\mathbb{F}_p$ be the scalar field of BLS12-381 (the group order). We write $[a]_1 = ag_1$ and $[a]_2 = ag_2$. The KHVC blinding scalar is $\rho \in \mathbb{F}_p$; RISE's ephemeral scalar is $r \in \mathbb{F}_p$; a file-commitment nonce is $\nu_i \in \{0, 1\}^{256}$. These three objects are never the same symbol.

Pairings. A Type-3 pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ on BLS12-381.

Assumption 1 (SXDH). *DDH is hard in $\mathbb{G}_1$ and in $\mathbb{G}_2$.*

Assumption 2 (q -SDH [1, 3]). *Given $([1]_1, [x]_1, \dots, [x^q]_1, [1]_2, [x]_2)$, a PPT adversary outputs $(c, [1/(x+c)]_1)$ with $c \neq -x$ only with negligible probability. The reduction uses this tuple as the KHVC SRS of degree $q = n$.*

KZG [3]. $\text{Setup}(1^\lambda, d)$ samples τ and publishes $([1]_1, \dots, [\tau^d]_1, [1]_2, [\tau]_2)$. $\text{Com}(f) = [f(\tau)]_1$. An opening of f at z is $[q(\tau)]_1$ for $q(X) = (f(X) - f(z))/(X - z)$. Evaluation binding follows from q -SDH. Commitments and openings are additively homomorphic. Verification uses only $[1]_2$ and $[\tau]_2$; it does not need $[\tau^d]_2$.

Definition 1 (HVC [12]). *A homomorphic vector commitment consists of $\text{Setup}(1^\lambda, n)$, $\text{Com}(\text{pp}, \mathbf{m}) \rightarrow (C, \text{aux})$, $\text{Open}(\text{pp}, \mathbf{m}, \text{aux}, i) \rightarrow \Lambda_i$, $\text{Ver}(\text{pp}, C, i, \mathbf{m}, \Lambda_i)$, $\text{ComHom}(C, C')$, $\text{OpenHom}(\Lambda_i, \Lambda'_i)$. The auxiliary string aux is independent of n . Position binding: no PPT adversary outputs $C, i, m \neq m', \Lambda, \Lambda'$ that both verify. Position hiding: given C and $\{\Lambda_j\}_{j \neq i}$ as group elements, the slot- i message is statistically hidden (Theorem 2). Homomorphism: $C \oplus C'$ and $\Lambda_i \otimes \Lambda'_i$ open to the coordinate-wise sum.*

RISE [4]. ElGamal-style UE in a prime-order group: $\text{Enc}_k(M) = (rY, rG + M)$ with $Y = kG$, token $\Delta = k'/k$, update re-randomizes and multiplies C_1 by Δ . Lehmann–Tackmann prove IND-ENC and rand-IND-UPD from DDH in that group. We instantiate the group as $\mathbb{G}_1$; SXDH is the Type-3 stand-in for that DDH assumption. RISE is additively homomorphic on the message group, which is what USS needs for encrypted HVC openings. It is *not* a large-file cipher: the message is a group element.

USS [12]. A database of n files. **Store** produces a repository of ciphertexts and a stub C . **Rev** returns file i or $\perp$. **FileUp** replaces file i ; **KeyUp** rotates keys. Security games, inherited verbatim from [12]: IND-DD-UP, IND-FileUp-CPA, IND-REENC-CPA, and OF-PTXT (integrity against a cheating server who may know the secret key). Section 1 lists the hypotheses under which we instantiate those games; Section 5 gives the hops.

Remark 1 (Instantiation hypotheses). *The rest of the paper uses: (i) Type-3 BLS12-381 and SXDH; (ii) q -SDH with $q = n$ for KHVC binding; (iii) a trusted KZG SRS (knowledge of τ is forbidden to all parties after setup); (iv) SHA-256 modelled as a random oracle, both as COM and as the ring elements $a_b = H(\text{tweak}||b)$; (v) file nonces ν_i of 256 bits of entropy; (vi) RISE in $\mathbb{G}_1$, IND-ENC and rand-IND-UPD under SXDH [4]; (vii) decisional Ring-LWR in dimension $N = 1024$, $R_q = \mathbb{Z}_q[X]/(X^N + 1)$, $q = 998244353$, rounding modulus $p = 2^{16}$, and a uniform secret in R_q . The DEM is correct for at most 127 updates: each update adds at most 1 to the rounding error, and eight noise bits absorb an error of absolute value at most 127.*

3 KHVC: Linear-Size Homomorphic Vector Commitment

A Homomorphic Vector Commitment (HVC) [12] allows a client to commit to a vector of messages $\vec{m} = (m_0, \dots, m_{n-1}) \in \mathbb{F}_p^n$ via a succinct string C , generate openings Λ_i proving that m_i is the

i -th entry, and dynamically update C and Λ_i as vector elements change. In the context of USS, HVC forms the core of the client’s local stub to enforce version freshness.

However, existing HVC instantiations impose severe system overhead. The standard construction proposed by Catalano and Fiore (CF-HVC) [2] relies on precomputing all pairwise group products $g^{z_i z_j}$, resulting in a Common Reference String (CRS) of size $O(n^2)$. As n scales to tens of thousands of files, this quadratic growth causes parameter generation times to explode into minutes and parameter sizes into gigabytes, creating a critical bottleneck for system startup and update operations.

Design Strategy & Technical Intuition. To overcome the quadratic parameter explosion while retaining strong position hiding and binding, we design KHVC, a succinct $O(n)$ -size HVC built upon the Kate–Zaverucha–Goldberg (KZG) polynomial commitment framework [3]. Our optimization is driven by three key insights:

1. **Linear-Size SRS via Roots of Unity:** We evaluate polynomials over the domain $\Omega = \{\omega^0, \dots, \omega^{n-1}\}$, defined by the n -th roots of unity, with the vanishes-on- Ω polynomial $Z(X) = X^n - 1$. Under this setting, standard KZG opening proofs need only a degree- n Structured Reference String (SRS) consisting of $n + 1$ elements in $\mathbb{G}_1$ and 2 elements in $\mathbb{G}_2$, cutting SRS complexity from $O(n^2)$ down to $O(n)$.
2. **Zero-Overhead Statistical Position Hiding:** To satisfy the privacy requirement that an opening Λ_i reveals no information about other vector slots $m_{j \neq i}$, we inject a random blinding scalar $\rho \leftarrow \mathbb{F}_p$ by defining $F(X) = f(X) + \rho Z(X)$, where $f(\omega^i) = m_i$. Since $Z(\omega^i) = 0$ for all $\omega^i \in \Omega$, adding $\rho Z(X)$ leaves evaluation results $F(\omega^i) = m_i$ completely intact. Consequently, position hiding is achieved statistically via a single scalar ρ , eliminating the need for additional $\mathbb{G}_2$ elements in the SRS.
3. **One-Hot Closed Form for Fast Updates:** Instead of performing dense $O(n \log n)$ FK20 proofs [15] over all n slots during single-slot file updates (FileUp), we precompute a key update table $W_j = [Z(\tau)/(\tau - \omega^j)]_1$ via a single $\mathbb{G}_1$ FFT during setup. We derive a closed-form formula for one-hot vector deltas, reducing off-slot opening updates to simple $O(1)$ scalar multiplications.

3.1 HVC Definitions

Syntax. A Homomorphic Vector Commitment (HVC) scheme for vectors of dimension n consists of six algorithms:

- **HVC.Setup($1^\lambda, n$) $\rightarrow$ pp:** Takes as input the security parameter 1^λ and vector size n , and outputs public parameters **pp**.
- **HVC.Com(pp, $\vec{m}$) $\rightarrow$ (C , aux):** Takes as input **pp** and a message vector $\vec{m} = (m_0, \dots, m_{n-1}) \in \mathcal{M}^n$, and outputs a succinct commitment C along with auxiliary information **aux** (where $|\mathbf{aux}|$ is independent of n).
- **HVC.Open(pp, $\vec{m}$, aux, i) $\rightarrow$ Λ_i :** Takes as input **pp**, vector $\vec{m}$, auxiliary string **aux**, and index $i \in \{0, \dots, n-1\}$, and outputs an opening proof Λ_i proving that the i -th entry is m_i .
- **HVC.Ver(pp, C , i , m_i , Λ_i) $\rightarrow$ 1/0:** Takes as input **pp**, commitment C , index i , message m_i , and proof Λ_i , and outputs 1 if the proof is valid, or 0 otherwise.
- **HVC.ComHom(pp, C , C') $\rightarrow$ C_{sum} :** Takes as input two commitments C, C' and outputs a homomorphically combined commitment C_{sum} .
- **HVC.OpenHom(pp, Λ_i , Λ'_i) $\rightarrow$ $\Lambda_{i,\text{sum}}$:** Takes as input two opening proofs Λ_i, Λ'_i at the same index i , and outputs a homomorphically combined proof $\Lambda_{i,\text{sum}}$.

Security Definitions. An HVC scheme must satisfy position binding and statistical position hiding:

- **Position Binding:** For all PPT adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ produces a commitment C , index i , and two distinct messages $m \neq m'$ with valid proofs Λ, Λ' is negligible:

$$\Pr [m \neq m' \wedge \text{HVC.Ver}(\text{pp}, C, i, m, \Lambda) = 1 \wedge \text{HVC.Ver}(\text{pp}, C, i, m', \Lambda') = 1] \leq \text{negl}(\lambda).$$

- **Statistical Position Hiding:** For target index i^* and any two vectors $\vec{m}_0, \vec{m}_1$ that agree on all slots $j \neq i^*$, the distributions of $(C_0, \{\Lambda_{0,j}\}_{j \neq i^*})$ and $(C_1, \{\Lambda_{1,j}\}_{j \neq i^*})$ are statistically indistinguishable.

3.2 Concrete Construction

Let $\Omega = \{\omega^0, \dots, \omega^{n-1}\}$ be the n -th roots of unity, $Z(X) = X^n - 1$, and let $\mathbb{F}_p$ be the scalar field of BLS12-381.

$\text{HVC.Setup}(1^\lambda, n) \rightarrow \text{pp}$. Sample $\tau \leftarrow \mathbb{F}_p^*$ uniformly at random. Publish the degree- n KZG SRS as part of pp :

$$\text{SRS} = ([1]_1, [\tau]_1, \dots, [\tau^n]_1, [1]_2, [\tau]_2) \in \mathbb{G}_1^{n+1} \times \mathbb{G}_2^2.$$

Note that $[\tau^n]_2$ is not required because verification only involves $[\tau]_2 - [\omega^i]_2$. The hiding shift $[Z(\tau)]_1 = [\tau^n]_1 - [1]_1$ is obtained for free from the SRS. Precompute the table $W_j = [Z(\tau)/(\tau - \omega^j)]_1$ via a single $\mathbb{G}_1$ FFT of the SRS (data-independent, adding n compressed $\mathbb{G}_1$ elements). Public parameters are $\text{pp} = (\text{SRS}, \{W_j\}_{j=0}^{n-1})$.

At $n = 1024$, the verification SRS is $(n+1) \cdot 48 + 2 \cdot 96 + 48 = 48.3$ KB, which is $\approx 1070\times$ smaller than Catalano–Fiore (50.5 MB). Operational parameters, including the precomputed table $\{W_j\}$, total $48.3 + 48 = 96.3$ KB (a $512\times$ raw size reduction).

$\text{HVC.Com}(\text{pp}, \vec{m}) \rightarrow (C, \text{aux})$. Interpolate $f(X) \in \mathbb{F}_p[X]$ of degree $< n$ such that $f(\omega^i) = m_i$ for all $i \in \{0, \dots, n-1\}$. Sample a random blinding factor $\rho \leftarrow \mathbb{F}_p$. Construct $F(X) = f(X) + \rho Z(X)$ and compute the commitment:

$$C = [F(\tau)]_1 = [f(\tau)]_1 + \rho[Z(\tau)]_1.$$

Output $(C, \text{aux} = \rho)$. The client retains ρ privately; the server never receives it.

$\text{HVC.Open}(\text{pp}, \vec{m}, \text{aux}, i) \rightarrow \Lambda_i$. Compute the quotient polynomial $q_i(X) = (F(X) - m_i)/(X - \omega^i)$ and output the opening proof $\Lambda_i = [q_i(\tau)]_1$.

$\text{HVC.Ver}(\text{pp}, C, i, m_i, \Lambda_i) \rightarrow 1/0$. Check if the following pairing equation holds:

$$e(C - [m_i]_1, [1]_2) = e(\Lambda_i, [\tau]_2 - [\omega^i]_2).$$

$\text{HVC.ComHom}(\text{pp}, C, C') \rightarrow C_{\text{sum}}$ & $\text{HVC.OpenHom}(\text{pp}, \Lambda_i, \Lambda'_i) \rightarrow \Lambda_{i,\text{sum}}$. KHVC inherits additive homomorphism directly from the underlying KZG framework:

$$\text{HVC.ComHom}(\text{pp}, C, C') = C + C', \quad \text{HVC.OpenHom}(\text{pp}, \Lambda_i, \Lambda'_i) = \Lambda_i + \Lambda'_i.$$

Re-randomization can be executed via $\text{ReRand}(C) = C + \rho'[Z(\tau)]_1$, which updates the blinding scalar without altering any vector evaluations $F(\omega^i)$.

3.3 Formal Security Proofs

Theorem 1 (Position Binding). *If the q -SDH assumption holds with $q = n$, then KHVC satisfies position binding. An adversary with advantage ε yields a q -SDH solver with advantage $\varepsilon - O(1/p)$.*

Proof. Suppose a PPT adversary $\mathcal{A}$ outputs a commitment C , index i , and two distinct openings (m, Λ) and (m', Λ') for $m \neq m'$ such that both pass verification:

$$e(C - [m]_1, [1]_2) = e(\Lambda, [\tau]_2 - [\omega^i]_2), \quad e(C - [m']_1, [1]_2) = e(\Lambda', [\tau]_2 - [\omega^i]_2).$$

Subtracting the two equations gives $e([m' - m]_1, [1]_2) = e(\Lambda - \Lambda', [\tau]_2 - [\omega^i]_2)$. By bilinear properties, this simplifies to:

$$[m' - m]_1 = (\tau - \omega^i)(\Lambda - \Lambda').$$

Since $m \neq m'$, $(m' - m)$ is invertible in $\mathbb{F}_p$ except with probability $O(1/p)$. Multiplying both sides by $(m' - m)^{-1}$ yields:

$$\left[\frac{1}{\tau - \omega^i} \right]_1 = (m' - m)^{-1}(\Lambda - \Lambda').$$

Given a q -SDH challenge instance $([1]_1, [\tau]_1, \dots, [\tau^n]_1, [1]_2, [\tau]_2)$, the reduction sets $c = -\omega^i$ and outputs $(c, [1/(\tau + c)]_1)$ as a valid solution to q -SDH with advantage $\varepsilon - O(1/p)$. $\square$

Theorem 2 (Statistical Position Hiding). *Let the adversary view C and $\{\Lambda_j\}_{j \neq i^*}$ as $\mathbb{G}_1$ elements. Except with probability at most n/p over setup (the event $\tau \in \Omega$), KHVC satisfies statistical position hiding with advantage at most n/p .*

Proof. The adversary chooses a target index i^* and two message vectors $\vec{m}_0, \vec{m}_1$ that agree on all slots $j \neq i^*$. Let $\Delta m = m_{i^*,1} - m_{i^*,0}$, and let $L_{i^*}(X)$ be the i^* -th Lagrange basis polynomial over Ω . The interpolated polynomials satisfy $f_1(X) = f_0(X) + \Delta m \cdot L_{i^*}(X)$.

Provided $\tau \notin \Omega$, $Z(\tau) \neq 0$. For any blinding factor $\rho_0 \in \mathbb{F}_p$ in world 0, define $\rho_1 = \rho_0 - \Delta m \cdot \frac{L_{i^*}(\tau)}{Z(\tau)}$. Evaluating $F_1(\tau)$ yields:

$$F_1(\tau) = f_1(\tau) + \rho_1 Z(\tau) = f_0(\tau) + \Delta m L_{i^*}(\tau) + \left(\rho_0 - \Delta m \frac{L_{i^*}(\tau)}{Z(\tau)} \right) Z(\tau) = F_0(\tau).$$

Hence, $C_1 = [F_1(\tau)]_1 = [F_0(\tau)]_1 = C_0$ is identically distributed in both worlds.

For any unrevealed opening index $j \neq i^*$, the opening element is $\Lambda_{b,j} = [(F_b(\tau) - m_{b,j})/(\tau - \omega^j)]_1$. Because $m_{0,j} = m_{1,j}$ for all $j \neq i^*$ and $F_0(\tau) = F_1(\tau)$, we have $\Lambda_{0,j} = \Lambda_{1,j}$.

Since $\rho_0 \mapsto \rho_1$ is a uniform bijection over $\mathbb{F}_p$, the joint distribution $(C_0, \{\Lambda_{0,j}\}_{j \neq i^*})$ and $(C_1, \{\Lambda_{1,j}\}_{j \neq i^*})$ seen by the adversary in $\text{Exp}_{\mathcal{A}, \text{HVC}}^{\text{PosHide}}$ are identical. Thus, an unbounded distinguisher has advantage at most n/p . $\square$

3.4 Efficiency Lemmas and Fast Updates

Lemma 1 (All-Openings Decomposition). *Let $F = f + \rho Z$ with $\deg f < n$ and $m_i = F(\omega^i) = f(\omega^i)$. Then:*

$$\frac{F(X) - m_i}{X - \omega^i} = \frac{f(X) - m_i}{X - \omega^i} + \rho \cdot \frac{Z(X)}{X - \omega^i}.$$

Equivalently, running FK20 [15] on the length- n prefix $(f_0 - \rho, f_1, \dots, f_{n-1})$ computes all baseline openings π_i^{FK20} , yielding $\Lambda_i = \pi_i^{\text{FK20}} + \rho \cdot W_i$, where $W_i = [Z(\tau)/(\tau - \omega^i)]_1$. KeyUp operations opening zero vectors reduce to n scalar multiplications over precomputed W_j , avoiding $O(n^2)$ naive MSMs.

Lemma 2 (One-Hot FileUp Openings). *Let $\Delta \in \mathbb{F}_p$, $i \in \{0, \dots, n-1\}$, and $f = \Delta L_i$ where $L_i(X) = \frac{\omega^i}{n} \frac{Z(X)}{X - \omega^i}$. Write $F = f + \rho Z$. For every $j \neq i$, the opening update is given in closed form by:*

$$\Lambda_j = \Delta \cdot \frac{\omega^i}{n} \cdot \frac{W_i - W_j}{\omega^i - \omega^j} + \rho \cdot W_j.$$

The opening at slot i is computed via a single length- n MSM. This avoids running dense FK20 algorithms when a single-slot vector update (FileUp) occurs.

4 Efficient IND-CPA Updatable Encryption

To handle large file payloads efficiently without relying on AES-GCM (which does not support direct ciphertext-independent re-encryption), we employ an efficient Data Encapsulation Mechanism (DEM) instantiated as a symmetric-key Updatable Encryption (UE) scheme. This construction is inspired by the Ring-LWR based updates used in CGLT23 [12] as well as the lightweight symmetric UE paradigm of SHINE/OCBSHINE [6] and ReCrypt [5].

4.1 Syntax and Security Definitions of UE

We use the ciphertext-independent sequential model of Lehmann and Tackmann [4], specialized to a symmetric DEM (no public key). Keys evolve by epochs $e = 0, 1, 2, \dots$; a single update token moves every ciphertext from epoch $e - 1$ to epoch e .

Syntax. A symmetric-key updatable encryption scheme UE consists of six algorithms:

- $\text{UE.Setup}(1^\lambda) \rightarrow \text{pp}$: outputs public parameters.
- $\text{UE.KeyGen}(\text{pp}) \rightarrow s_0$: samples the initial epoch secret key.
- $\text{UE.NextKey}(s_e) \rightarrow (s_{e+1}, \Delta_{e+1})$: samples the next epoch key and a ciphertext-independent update token.
- $\text{UE.Enc}(s_e, \text{tweak}, m) \rightarrow \text{ct}$: encrypts m under the current key (and an optional public tweak).
- $\text{UE.Dec}(s_e, \text{tweak}, \text{ct}) \rightarrow m/\perp$: decrypts.
- $\text{UE.ReEnc}(\Delta_{e+1}, \text{ct}) \rightarrow \text{ct}'$: updates a ciphertext from epoch e to epoch $e + 1$ using only the token.

Correctness requires $\text{UE.Dec}(s_e, \text{tweak}, \text{UE.Enc}(s_e, \text{tweak}, m)) = m$ and, after any sequence of honest updates of length at most the scheme's noise budget, $\text{UE.Dec}(s_{e+t}, \text{tweak}, \text{UE.ReEnc}^*(\text{UE.Enc}(s_e, \text{tweak}, m))) = m$.

Oracles (following [4, §3.2]). In the security experiments the challenger maintains an epoch counter e , the current key s_e , the current token Δ_e , a challenge epoch $\tilde{e}$ (initially $\perp$), and lists

- $\mathcal{L}$: honest non-challenge ciphertexts (c, e) produced by $\mathcal{O}^{\text{Enc}}$ or $\mathcal{O}^{\text{ReEnc}}$;
- $\tilde{\mathcal{L}}$: updated versions of the challenge ciphertext;
- $\mathcal{C}$: epochs in which the adversary obtained a challenge ciphertext (via $\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$);
- $\mathcal{K}$: epochs in which the adversary corrupted a secret key;
- $\mathcal{T}$: epochs in which the adversary corrupted an update token.

The adversary may query:

$\mathcal{O}^{\text{Enc}}(m)$

return $c \leftarrow \text{UE.Enc}(s_e, \text{tweak}, m)$, add (c, e) to $\mathcal{L}$.

$\mathcal{O}^{\text{Next}}$

run $(s_{e+1}, \Delta_{e+1}) \leftarrow \text{UE.NextKey}(s_e)$ and set $e \leftarrow e + 1$; if the challenge has already been issued, also update the challenge ciphertext $\tilde{c}_e \leftarrow \text{UE.ReEnc}(\Delta_e, \tilde{c}_{e-1})$ and store it in $\tilde{\mathcal{L}}$.

$\mathcal{O}^{\text{ReEnc}}(c)$

require $(c, e - 1) \in \mathcal{L}$; return $c' \leftarrow \text{UE.ReEnc}(\Delta_e, c)$ and add (c', e) to $\mathcal{L}$.

$\mathcal{O}^{\text{Corrupt}}(\text{key}, e^*) / \mathcal{O}^{\text{Corrupt}}(\text{token}, e^*)$

for $e^* \leq e$, return s_{e^*} (add e^* to $\mathcal{K}$) or Δ_{e^*} (add e^* to $\mathcal{T}$), respectively.

$\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$

return the current challenge ciphertext $\tilde{c}_e \in \tilde{\mathcal{L}}$ and add e to $\mathcal{C}$.

Extended sets $\mathcal{C}^*, \mathcal{K}^*, \mathcal{T}^*$ close $\mathcal{C}, \mathcal{K}, \mathcal{T}$ under the scheme's leakage profile (token inference and uni-/bi-directional updates) as in [4, §3.3]. A trivial win occurs if $\mathcal{C}^* \cap \mathcal{K}^* \neq \emptyset$ (the adversary knows a challenge ciphertext and the matching epoch key).

Definition 2 (IND-ENC-CPA [4]). *An updatable encryption scheme UE is IND-ENC-CPA secure if for every PPT adversary $\mathcal{A}$,*

$$\left| \Pr[\text{Exp}_{\mathcal{A}, \text{UE}}^{\text{IND-ENC-CPA}}(\lambda) = 1] - \frac{1}{2} \right|$$

is negligible in λ , where the experiment proceeds as follows.

1. *The challenger samples $s_0 \leftarrow \text{UE.KeyGen}(\text{UE.Setup}(1^\lambda))$, sets $e \leftarrow 0$, $\tilde{e} \leftarrow \perp$, and initializes empty lists $\mathcal{L}, \tilde{\mathcal{L}}, \mathcal{C}, \mathcal{K}, \mathcal{T}$.*
2. *$\mathcal{A}$ may adaptively query $\mathcal{O}^{\text{Enc}}$, $\mathcal{O}^{\text{Next}}$, $\mathcal{O}^{\text{ReEnc}}$, and $\mathcal{O}^{\text{Corrupt}}$.*
3. *$\mathcal{A}$ outputs a challenge (m_0, m_1, st) with $|m_0| = |m_1|$. The challenger samples $b \leftarrow \{0, 1\}$, sets $\tilde{e} \leftarrow e$, $\tilde{c} \leftarrow \text{UE.Enc}(s_{\tilde{e}}, \text{tweak}, m_b)$, stores $(\tilde{c}, \tilde{e})$ in $\tilde{\mathcal{L}}$, and returns $\tilde{c}$ to $\mathcal{A}$.*
4. *$\mathcal{A}$ continues with the same oracles plus $\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$, then outputs a bit b' .*
5. *The experiment returns 1 iff $b' = b$ and $\mathcal{C}^* \cap \mathcal{K}^* = \emptyset$ (no secret-key corruption in any challenge-equal epoch).*

This is the standard CPA game lifted to sequential key rotation with adaptive key/token corruption and post-compromise security [4]: fresh encryptions hide the plaintext even if keys before/after the challenge epoch leak, provided the adversary never obtains both a challenge ciphertext and its epoch key.

Definition 3 (IND-UPD-CPA [4]). *An updatable encryption scheme UE is IND-UPD-CPA secure if for every PPT adversary $\mathcal{A}$,*

$$\left| \Pr[\text{Exp}_{\mathcal{A}, \text{UE}}^{\text{IND-UPD-CPA}}(\lambda) = 1] - \frac{1}{2} \right|$$

is negligible in λ , where the experiment proceeds as follows.

1. *Setup is identical to Definition 2.*
2. *$\mathcal{A}$ queries $\mathcal{O}^{\text{Enc}}$, $\mathcal{O}^{\text{Next}}$, $\mathcal{O}^{\text{ReEnc}}$, and $\mathcal{O}^{\text{Corrupt}}$.*

3. $\mathcal{A}$ outputs two ciphertexts (c_0, c_1, st) that both appear in $\mathcal{L}$ at epoch $e - 1$ and satisfy $|c_0| = |c_1|$. The challenger samples $b \leftarrow \{0, 1\}$, sets $\tilde{e} \leftarrow e$, $\tilde{c} \leftarrow \text{UE.ReEnc}(\Delta_{\tilde{e}}, c_b)$, stores $(\tilde{c}, \tilde{e})$ in $\tilde{\mathcal{L}}$, and returns $\tilde{c}$ to $\mathcal{A}$.
4. $\mathcal{A}$ continues with the same oracles plus $\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$, then outputs b' .
5. The experiment returns 1 iff $b' = b$ and all of the following hold:
 - (a) $\tilde{e} \notin \mathcal{T}^*$ (the challenge-epoch token was not corrupted);
 - (b) $\mathcal{C}^* \cap \mathcal{K}^* = \emptyset$ (no key corruption in a challenge-equal epoch);
 - (c) if UE.ReEnc is deterministic, then $\mathcal{A}$ did not query $\mathcal{O}^{\text{ReEnc}}(c_0)$ or $\mathcal{O}^{\text{ReEnc}}(c_1)$ in epoch $\tilde{e}$.

IND-UPD-CPA is the unlinkability game of [4]: an updated ciphertext hides which of two equal-length prior ciphertexts it came from, even under adaptive corruptions outside the trivial-win region. (If updates are randomized, as for RISE and our Ring-LWR DEM, condition (c) is vacuous.)

Remark 2 (DEM specialization used in USS). *In the USS hybrids of Section 5 the body UE is only accessed as a CPA DEM: the reduction never corrupts the challenge-epoch DEM secret, and KeyUp is the honest NextKey/ReEnc path. Remark 3 is exactly the IND-ENC-CPA hop (uniform bodies) and the IND-UPD-CPA hop (updated bodies $\approx$ fresh encryptions under s') under that restriction. Full adaptive post-compromise security of the Ring-LWR DEM as a standalone UE is Theorem 3 (citing [9, 8, 7]), with at most 127 updates inside the noise pad.*

4.2 Concrete Construction from Ring-LWR

Let $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ and $R_p = \mathbb{Z}_p[X]/(X^N + 1)$, where $N = 1024$, q is a 32-bit prime, and p is a scalar field modulus matching the HVC framework. Let $H : \{0, 1\}^* \rightarrow R_q$ be a hash function modeled as a Random Oracle.

UE.Setup(1^λ). Output $\text{pp} = (N, q, p, H)$.

UE.KeyGen(pp). Sample a uniform secret $s \leftarrow R_q$.

UE.Enc(s, tweak, m). For each block index b of message $m \in R_p$:

1. Compute $a_b = H(\text{tweak} \parallel b) \in R_q$.
2. Compute pseudo-random pad $F_s(b) = \left\lfloor \frac{p}{q}(a_b \cdot s) \right\rfloor \in R_p$.
3. Output ciphertext block $c_b = m_b + F_s(b) \pmod{p}$.

In implementation, a_b is computed in the twisted NTT domain and cached per file block alongside tweak .

UE.Dec(s, tweak, c_b). Recompute $a_b = H(\text{tweak} \parallel b)$, evaluate $F_s(b) = \left\lfloor \frac{p}{q}(a_b \cdot s) \right\rfloor \in R_p$, and recover $m_b = c_b - F_s(b) \pmod{p}$.

UE.ReEnc(Δ, c_b). Given the key transition token $\Delta = s' - s \in R_q$ and cached NTT of a_b :

1. Evaluate $F_\Delta(b) = \left\lfloor \frac{p}{q}(a_b \cdot \Delta) \right\rfloor \in R_p$.
2. Homomorphically update the ciphertext: $c'_b = c_b + F_\Delta(b) \pmod{p}$.

Noise Accumulation and Decryption Bound. Updating directly in $\mathbb{Z}_p$ introduces a rounding error of infinity norm $\|e\|_\infty \leq 1$ per rotation turn. After t consecutive updates, $\|e^{(t)}\|_\infty \leq t$. By reserving noise bits in each coefficient encoding, decryption is exact for up to $t \leq 127$ updates.

Theorem 3 (Ring-LWR DEM confidentiality). *Assume decisional Ring-LWR on R_q with rounding modulus p is hard, and model H as a random oracle. Let UE be the scheme above, restricted to at most $t \leq 127$ honest updates (so the noise pad never overflows). Then:*

1. $F_s(b) = \lfloor (p/q)(a_b \cdot s) \rfloor$ is a secure almost key-homomorphic PRF in the ROM [9, 8] (as recalled for UE in [7, §6]).
2. UE is IND-ENC-CPA secure (Definition 2).
3. UE is IND-UPD-CPA secure (Definition 3).

In particular, a fresh body is indistinguishable from uniform in R_p , and an honestly updated body is indistinguishable from a fresh encryption under the new secret s' (up to a message-independent rounding error of infinity norm at most t).

Proof. PRF security (prior work). For each block index b , the public ring element $a_b = H(\text{tweak}||b)$ is a uniformly random oracle output. The map $s \mapsto \lfloor (p/q)(a_b \cdot s) \rfloor$ is the standard Ring-LWR sample. Banerjee–Peikert–Rosen [9] and Boneh–Lewi–Montgomery–Raghunathan [8] prove that the keyed function $F_s(x) = \lfloor (p/q)(H(x) \cdot s) \rfloor$ is a secure PRF (and γ -almost key-homomorphic with $\gamma = 1$ for a single sample) in the random-oracle model under decisional (Ring-)LWR. Boneh–Eskandarian–Kim–Fisch [7, §6] explicitly recall this Ring-LWR almost KH-PRF as the lattice baseline for updatable encryption (and then replace it by RLWE only to enlarge the noise budget). We inherit that PRF theorem unchanged; our parameters merely fix a concrete 128-update pad.

Almost key-homomorphism. For any $s, s' \in R_q$ and $\Delta = s' - s$,

$$F_s(b) + F_\Delta(b) = \left\lfloor \frac{p}{q}(a_b \cdot s) \right\rfloor + \left\lfloor \frac{p}{q}(a_b \cdot \Delta) \right\rfloor = F_{s'}(b) + e_b$$

with a coefficient-wise rounding error $e_b \in \{-1, 0, 1\}^N$. Hence t successive updates accumulate $\|e\|_\infty \leq t$, which the 8-bit noise pad absorbs for $t \leq 127$.

IND-ENC-CPA. This is the standard “counter-mode from a PRF” argument, in the sequential UE game of Definition 2 (cf. the KH-PRF UE analyses of [8, 5] and the confidentiality reduction for [7, Construction 5.2]). Fix a PPT adversary $\mathcal{A}$. As long as $\mathcal{A}$ does not trigger a trivial win ($\mathcal{C}^* \cap \mathcal{K}^* = \emptyset$), every challenge-equal epoch key is unknown to $\mathcal{A}$.

- *Hybrid H_0 :* the real IND-ENC-CPA experiment.
- *Hybrid H_1 :* replace every evaluation of $F_{s_{\tilde{e}}}(\cdot)$ used to form the challenge ciphertext (and its later updates via $\mathcal{O}^{\text{Upd}\tilde{\mathcal{C}}}$, which only add public F_Δ -pads once Δ is known) by independent uniform elements of R_p .

Distinguishing H_0 from H_1 is exactly the PRF advantage against F under the unknown challenge-epoch secret (or, after an update into a new uncorrupted epoch, under that epoch’s secret), which is negligible by decisional Ring-LWR in the ROM. In H_1 the challenge ciphertext is m_b plus uniform pads, hence independent of the hidden bit b . Thus $\text{Adv}^{\text{IND-ENC-CPA}} \leq \text{Adv}_F^{\text{PRF}} \leq \text{Adv}^{\text{RLWR}}$.

IND-UPD-CPA. Again follow the almost-KH-PRF UE template [5, 7]. In Definition 3 the challenge is $\tilde{c} = \text{UE.ReEnc}(\Delta_{\tilde{e}}, c_b)$ for a hidden bit b , and $\tilde{e} \notin \mathcal{T}^*$ (the challenge-epoch token is not corrupted).

- *Hybrid G_0* : the real experiment.
- *Hybrid G_1* : write $c_b = \text{encode}(m) + \text{pad}_b$ for the (adversary-known) prior ciphertext. The challenge equals $\text{encode}(m) + F_{s_{\tilde{e}}}(\cdot) + e$ for a message-independent rounding error $\|e\|_{\infty} \leq 1$. Replace $F_{s_{\tilde{e}}}(\cdot)$ by a uniform pad in R_p (PRF / Ring-LWR under the unknown key $s_{\tilde{e}}$).
- *Hybrid G_2* : the challenge is now a uniform offset of a known string, independent of which prior ciphertext c_0 or c_1 was updated.

The hop $G_0 \rightarrow G_1$ is Ring-LWR; G_1 and G_2 are identical. Hence $\text{Adv}^{\text{IND-UPD-CPA}} \leq \text{Adv}^{\text{RLWR}}$. (The same hop shows that an updated body is indistinguishable from a fresh encryption under s' , up to the absorbed rounding error—the specialization used in Remark 3.)

What is not new. We do not claim a new lattice assumption or a new KH-PRF. The PRF theorem is [9, 8]; the reduction from almost KH-PRF counter-mode to UE confidentiality is the standard one used by [8, 5, 7]. What this paper contributes is the *instantiation and measurement* of the Ring-LWR DEM inside USS under a 127-update pad (whereas [7] abandoned Ring-LWR for RLWE to target a larger update budget). $\square$

5 Updatable Secure Storage (USS)

In this section, we present the formal syntax and security definitions for Updatable Secure Storage (USS), followed by our concrete construction using KHVC, RISE, and our Ring-LWR UE scheme, and complete security proofs.

5.1 USS Syntax and Security Definitions

Syntax. An Updatable Secure Storage (USS) scheme consists of six algorithms:

- $\text{USS.ParGen}(1^\lambda, n) \rightarrow \text{pp}$: Takes as input the security parameter 1^λ and maximum capacity n , outputting public parameters pp .
- $\text{USS.KeyGen}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$: Outputs client secret key sk and public key pk .
- $\text{USS.Store}(\text{pp}, \text{sk}, \vec{m}) \rightarrow (\text{stub}, \text{repo})$: Encrypts dataset $\vec{m} = (m_1, \dots, m_n)$, returning a local client stub stub and server repository repo .
- $\text{USS.Rev}(\text{pp}, \text{sk}, \text{repo}, i) \rightarrow m_i / \perp$: Decrypts and verifies slot i , returning m_i or failure symbol $\perp$.
- $\text{USS.FileUp}(\text{pp}, \text{sk}, \text{repo}, i, m'_i) \rightarrow (\text{stub}', \text{repo}')$: Updates slot i to m'_i with communication proportional to $|m'_i|$.
- $\text{USS.KeyUp}(\text{pp}, \text{sk}, \text{sk}', \text{repo}) \rightarrow (\text{token}, \text{repo}')$: Generates token token allowing server-side key rotation from sk to sk' .

Security Definitions. We adopt the four USS games of CGLT23 [12, §3.2] verbatim (only notation aligned with our algorithms). A challenge ciphertext is the encryption of a target file; after key rotation its image is called a *challenge-equal* ciphertext, and the repository containing them is the challenge-equal repository.

Challenger state [12]. The challenger maintains: e (current epoch, init. 1); e^* (challenge epoch, init. $\perp$); sb^* (stub of the challenge-equal repository, init. $\perp$); $\mathcal{K}$ (epochs where the adversary corrupted the epoch key); $\mathcal{C}$ (epochs where it corrupted a challenge-equal ciphertext); $\mathcal{I}$ (indices of challenge-equal ciphertexts; empty before the challenge); $\mathcal{L}$ (tuples $(t, \text{sb}, \text{repo}, \text{db})$ tracking all repositories); $\mathcal{S}$ (tuples (t, Trans) of key-update transcripts from epoch $t - 1$ to t); $\mathcal{F}$ (tuples $(\text{sb}, t, i, \text{ft}_i)$ of file-update transcripts on challenge-equal slots); $\mathcal{T}$ (epochs where the adversary queried a key-update transcript).

Confidentiality oracles [12].

$\mathcal{O}.\text{Store}(\text{db})$

run $(\text{sb}, \text{repo}) \leftarrow \text{Store}(\text{sk}_e, \text{db}, \text{pp})$, return (sb, repo) , add $(e, \text{sb}, \text{repo}, \text{db})$ to $\mathcal{L}$.

$\mathcal{O}.\text{Next}$

for every $(e, \text{sb}, \text{repo}, \text{db}) \in \mathcal{L}$ sample $\text{sk}' \leftarrow \text{KeyGen}(\text{pp})$ and run KeyUp to obtain $(\text{sb}', \text{repo}', \text{Trans})$; set $e \leftarrow e + 1$, append updated entries to $\mathcal{L}$ and (e, Trans) to $\mathcal{S}$; if $\text{sb} = \text{sb}^*$ then update $\text{sb}^* \leftarrow \text{sb}'$; return to $\mathcal{A}$ all updated $(\text{sb}', \text{repo}')$ for non-challenge repositories, and for the challenge-equal repository only the stub and ciphertexts with indices $j \notin \mathcal{I}$.

$\mathcal{O}.\text{KeyCorr}(t)$

if $t \leq e$, return sk_t and add t to $\mathcal{K}$.

$\mathcal{O}.\text{KeyUpTrans}(t)$

if $t \leq e$, return the transcript Trans of the update from epoch $t - 1$ to t and add t to $\mathcal{T}$.

$\mathcal{O}.\text{FileUp}(\text{sb}, m'_i, i)$

retrieve $(e, \text{sb}, \text{repo}, \text{db}) \in \mathcal{L}$; run FileUp to $(\text{sb}', \text{repo}')$; if $\text{sb} \neq \text{sb}^*$, return $(\text{sb}', \text{repo}', \text{ft}_i)$; if $\text{sb} = \text{sb}^*$, set $\text{sb}^* \leftarrow \text{sb}'$; if $i \in \mathcal{I}$ remove i from $\mathcal{I}$ and record $(\text{sb}, e, i, \text{ft}_i)$ in $\mathcal{F}$; return sb' and all updated ciphertexts with indices $j \notin \mathcal{I}$ (and ft_i if $i \notin \mathcal{I}$).

$\mathcal{O}.\text{ChaCTCorr}(j)$

if $j \in \mathcal{I}$ and $\text{sb}^* \neq \perp$, add e to $\mathcal{C}$ and return the j -th ciphertext of the challenge-equal repository; else $\perp$.

$\mathcal{O}.\text{ChaFTCorr}(\text{sb}, t, i)$

return the recorded challenge-equal file-update transcript ft_i if present.

Trivial win [12]. Key-update transcripts may update or downgrade challenge-equal ciphertexts (bi-directional leakage of our building blocks). Define the extended corruption set $\tilde{\mathcal{C}}$ by $i \in \tilde{\mathcal{C}}$ iff $i \in \mathcal{C}$, or $(i - 1 \in \mathcal{C} \wedge i \in \mathcal{T})$, or $(i + 1 \in \mathcal{C} \wedge i + 1 \in \mathcal{T})$. Any confidentiality experiment that ends with $\mathcal{K} \cap \tilde{\mathcal{C}} \neq \emptyset$ is a trivial win and returns $\perp$ (advantage ignored).

Definition 4 (IND-DD-UP [12]). Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}}(1^\lambda, \mathcal{M}, n, b)$:

1. $\text{pp} \leftarrow \text{ParGen}(1^\lambda, \mathcal{M}, n)$; initialize $e = 1$, $e^* = \text{sb}^* = \perp$ and empty $\mathcal{K}, \mathcal{C}, \mathcal{I}, \mathcal{L}, \mathcal{S}, \mathcal{T}, \mathcal{F}$; $\text{sk}_1 \leftarrow \text{KeyGen}(\text{pp})$.
2. $(\text{db}_0, \text{db}_1, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}}(\text{pp})$ with $\text{db}_b = (m_{b,1}, \dots, m_{b,n})$.
3. $(\text{sb}^*, \text{repo}^*) \leftarrow \text{Store}(\text{db}_b, \text{sk}_e, \text{pp})$; set $e^* \leftarrow e$; add $(e, \text{sb}^*, \text{repo}^*, \text{db}_b)$ to $\mathcal{L}$; for each i with $m_{0,i} \neq m_{1,i}$ add i to $\mathcal{I}$.
4. $b' \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}, \mathcal{O}.\text{ChaFTCorr}, \mathcal{O}.\text{ChaCTCorr}}(\text{st}, \text{repo}^*)$.
5. Return b' if $\mathcal{K} \cap \tilde{\mathcal{C}} = \emptyset$, else $\perp$.

USS is IND-DD-UP secure if for every PPT $\mathcal{A}$,

$$\text{Adv}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}} := |\Pr[\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}}(1^\lambda, \mathcal{M}, n, 0) = 1] - \Pr[\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-DD-UP}}(1^\lambda, \mathcal{M}, n, 1) = 1]|$$

is negligible. Intuitively: even with adaptive key/file/key-update corruptions (outside trivial wins), $\mathcal{A}$ cannot tell which of two equal-length databases was stored.

Definition 5 (IND-FileUp-CPA [12]). Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-FileUp-CPA}}(1^\lambda, \mathcal{M}, n, b)$:

1. Setup as in Definition 4.
2. $(\text{sb}, i, m_{0,i}, m_{1,i}, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}}(\text{pp})$. Abort if $(e, \text{sb}, \cdot, \cdot) \notin \mathcal{L}$, or $i \notin [1, n]$, or $|m_{0,i}| \neq |m_{1,i}|$.
3. Retrieve $(e, \text{sb}, \text{repo}, \text{db})$; set $e^* \leftarrow e$, $\mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$, $\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}$. Run FileUp of slot i to $m_{b,i}$, obtaining $(\text{sb}^*, \text{repo}^*)$ and transcript ft ; add $(e, \text{sb}^*, \text{repo}^*, \text{db}_b)$ to $\mathcal{L}$ where db_b replaces coordinate i by $m_{b,i}$.
4. $b' \leftarrow \mathcal{A}^{\dots, \mathcal{O}.\text{ChaFTCorr}, \mathcal{O}.\text{ChaCTCorr}}(\text{st}, \text{sb}^*, \text{repo}^*, \text{ft})$.
5. Return b' if $\mathcal{K} \cap \tilde{\mathcal{C}} = \emptyset$, else $\perp$.

USS is IND-FileUp-CPA secure if the analogous distinguishing advantage is negligible. Intuitively: a FileUp hides which of two equal-length replacement files was written, even if $\mathcal{A}$ knew the previous database.

Definition 6 (IND-REENC-CPA [12]). Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{IND-REENC-CPA}}(1^\lambda, \mathcal{M}, n, b)$:

1. Setup as above (epoch counter starts at the value used by [12]).
2. $(\text{sb}_0, \text{sb}_1, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}}(\text{pp})$.
3. Retrieve $(\text{sb}_0, \text{repo}_0, \text{db}_0)$ and $(\text{sb}_1, \text{repo}_1, \text{db}_1)$ from $\mathcal{L}$ at the current epoch; set $\mathcal{I} = \{i : c_{0,i} \neq c_{1,i}\}$, sample a fresh epoch key, set $e \leftarrow e + 1$, $e^* \leftarrow e$, $\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}$. Run KeyUp on repository b to obtain $(\text{sb}^*, \text{repo}^*)$; honestly KeyUp every other repository as well.
4. $b' \leftarrow \mathcal{A}^{\mathcal{O}.\text{Store}, \mathcal{O}.\text{Next}, \mathcal{O}.\text{FileUp}, \mathcal{O}.\text{KeyCorr}, \mathcal{O}.\text{KeyUpTrans}, \mathcal{O}.\text{ChaCTCorr}}(\text{st}, \text{sb}^*, \text{repo}^*, \{\text{all other updated } (\text{sb}', \text{repo}')\})$.
5. Return b' if $\mathcal{K} \cap \tilde{\mathcal{C}} = \emptyset$, else $\perp$.

USS is IND-REENC-CPA secure if the distinguishing advantage is negligible. Intuitively: after KeyUp, $\mathcal{A}$ cannot tell which of two previously stored repositories was rotated (unlinkability / post-compromise security for the whole repository), even with FileUp oracles.

Definition 7 (OF-PTXT [12]). The integrity challenger keeps a list $\mathcal{R}$ of the latest honest pairs (sb, db) (only the current epoch). The adversary (malicious server) gets:

$\mathcal{O}.\text{StoreINT}(\text{db})$

run Store, return (sb, repo) , add (sb, db) to $\mathcal{R}$.

$\mathcal{O}.\text{NextINT}$

advance the epoch, sample a new key, run the client side of KeyUp on every stub, return all new stubs and client transcripts Trans_c ; update every $(\text{sb}, \text{db}) \in \mathcal{R}$ to (sb', db) (plaintexts unchanged).

$\mathcal{O}.\text{FileUpINT}(m'_i, i, \text{sb})$

if $\text{sb} \in \mathcal{R}$, run the client FileUp protocol against the adversarial server (who may deviate arbitrarily); if the client accepts, update $\mathcal{R}$ to (sb', db') .

$\mathcal{O}.\text{KeyCorr}$

return the current epoch secret (integrity must survive key leakage).

Experiment $\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{OF-PTXT}}(1^\lambda, \mathcal{M}, n)$:

1. $\text{pp} \leftarrow \text{ParGen}$; initialize $\mathcal{R}$, e ; $\text{sk} \leftarrow \text{KeyGen}(\text{pp})$.
2. $(\text{sb}, i, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}.\text{StoreINT}, \mathcal{O}.\text{NextINT}, \mathcal{O}.\text{FileUpINT}, \mathcal{O}.\text{KeyCorr}}(\text{pp})$. Abort if $(\text{sb}, \cdot) \notin \mathcal{R}$ or $i \notin [1, n]$.
3. Run Rev for slot i against $\mathcal{A}$ (adversarial repository) and obtain m_i^* .
4. Return 1 iff for every $(\text{sb}, \text{db}) \in \mathcal{R}$ one has $m_i^* \neq \text{db}[i]$ (and Rev accepted); else 0.

USS is OF-PTXT secure if $\Pr[\text{Exp}_{\text{USS}, \mathcal{A}}^{\text{OF-PTXT}} = 1]$ is negligible for every PPT $\mathcal{A}$. Intuitively: even a malicious server that knows the current secret and may refuse to follow *FileUp/KeyUp* cannot make the client accept a plaintext that is not the latest honest version bound to the client's stub.

5.2 Concrete Construction

We build USS using KHVC (Section 3), RISE in $\mathbb{G}_1$, and the Ring-LWR UE scheme (Section 4). Let $\text{COM}(m_i; \nu_i) = \text{SHA-256}(m_i \parallel \nu_i)$ mapped into $\mathbb{F}_p$ as a little-endian scalar. In the ROM, COM is hiding (256-bit random nonce ν_i) and binding.

$\text{USS.ParGen}(1^\lambda, n)$. Run $\text{KHVC.Setup}(1^\lambda, n)$ to generate $\text{pp}_{\text{HVC}} = (\text{SRS}, \{W_j\}_{j=0}^{n-1})$. Output $\text{pp} = \text{pp}_{\text{HVC}}$.

$\text{USS.KeyGen}(\text{pp})$. Sample RISE key pair $(k, Y = kg_1)$ with $g_1 \in \mathbb{G}_1$ and Ring-LWR secret $s \in R_q$. Output $\text{sk} = (k, s)$ and $\text{pk} = Y$.

$\text{USS.Store}(\text{pp}, \text{sk}, (m_1, \dots, m_n))$. For each $i \in \{1, \dots, n\}$:

1. Sample 256-bit nonce ν_i and fresh tweak_i . Compute commitment $h_i = \text{COM}(m_i; \nu_i) \in \mathbb{F}_p$.
2. Encrypt payload $\nu_i \parallel m_i$ as $C_i^{\text{UE}} \leftarrow \text{UE.Enc}(s, \text{tweak}_i, \nu_i \parallel m_i)$ and cache a_b NTTs.
3. Compute KHVC opening proof Λ_i for $(h_1, \dots, h_n)$, and RISE-encrypt it: $E_i = \text{RISE.Enc}_k(\Lambda_i)$.

Sample fresh scalar $\rho \in \mathbb{F}_p$. Compute $(C, \rho) \leftarrow \text{KHVC.Com}(\text{pp}, (h_1, \dots, h_n))$. The client keeps local stub $\text{stub} = C$, discards ρ , and sends repository $\text{repo} = (\{E_i\}, \{C_i^{\text{UE}}\}, \{\text{tweak}_i\})$ to the server.

$\text{USS.Rev}(\text{pp}, \text{sk}, \text{repo}, i)$. The client downloads E_i, C_i^{UE} , and tweak_i . It decrypts $\Lambda_i \leftarrow \text{RISE.Dec}_k(E_i)$, decrypts payload $(\nu_i, m_i) \leftarrow \text{UE.Dec}(s, \text{tweak}_i, C_i^{\text{UE}})$, and recomputes $h_i = \text{COM}(m_i; \nu_i)$. It returns m_i if $\text{KHVC.Ver}(\text{pp}, C, i, h_i, \Lambda_i) = 1$, else $\perp$.

$\text{USS.FileUp}(\text{pp}, \text{sk}, \text{repo}, i, m'_i)$. Client downloads slot i , extracts (m_i, ν_i) , samples fresh ν'_i , and computes $h'_i = \text{COM}(m'_i; \nu'_i)$. It sets one-hot delta $\Delta h_i = h'_i - h_i$ at index i . Sampling $\rho_\delta \in \mathbb{F}_p$, it sends delta commitment $C_\delta = [\Delta h_i L_i(\tau)]_1 + \rho_\delta [Z(\tau)]_1$ and fresh UE encryption $(C_i^{\text{UE}})', \text{tweak}'_i$ to the server. The server updates stub $\text{stub}' = \text{stub} + C_\delta$, updates opening proofs via one-hot closed form (Lemma 2), and homomorphically adds RISE encryptions.

$\text{USS.KeyUp}(\text{pp}, \text{sk}, \text{sk}', \text{repo})$. Let $\text{sk} = (k, s)$ and $\text{sk}' = (k', s')$. The client sends token $\text{token} = (\Delta_R = k'/k, Y' = k'g_1, \rho_0 \in \mathbb{F}_p, \Delta = s' - s)$. The server:

1. Updates RISE headers using Δ_R and adds $\text{RISE.Enc}_{k'}(\rho_0 \cdot W_j)$ to each opening.
2. Updates local stub: $\text{stub}' = \text{stub} + \rho_0 [Z(\tau)]_1$.
3. Re-encrypts payload bodies via $\text{UE.ReEnc}(\Delta, C_i^{\text{UE}})$ using cached a_b NTTs.

5.3 Security Proofs

We prove confidentiality by hybrid arguments aligned with [12, Thms. 3–5], under the substitutions of Remark 1: RISE in $\mathbb{G}_1$ (SXDH), Type-3 pairing checks, KHVC in place of Catalano–Fiore, and the Ring-LWR UE of Section 4 as the DEM. Integrity (OF-PTXT) is Lemma 6.

Theorem 4 (USS Security). *Under Remark 1, and writing Adv for the USS advantages of [12]:*

1. KHVC is statistically position hiding (Theorem 2) and tightly q -SDH position binding.
2. IND-DD-UP, IND-FileUp-CPA, and IND-REENC-CPA hold as proved below.
3. OF-PTXT holds by Lemma 6, even if the current RISE key, DEM secret, and every KeyUp nonce ρ_0 leak.
4. A fresh body (and a body after at most 127 updates) is IND-CPA under decisional Ring-LWR in the ROM. The DEM is not authenticated encryption.

Remark 3 (DEM in the random oracle). *Let H be the random oracle used as COM and as $a_b = H(\text{tweak} \| b)$. A body block is of the form $\text{encode}(\nu \| m) + F_s(b)$ with $F_s(b) = \lfloor (p/q)(a_b \cdot s) \rfloor \in R_p$. Replacing every block of a challenge body by an independent uniform element of R_p is a standard decisional Ring-LWR hop for the epoch secret s : each a_b is a fresh oracle string (fresh tweak, or a previously unused block index), so the rounded products are Ring-LWR samples. The same hop applies after KeyUp. The server receives $\Delta = s' - s$. Since s' is uniform in R_q , Δ is uniform and independent of s , and an honestly updated block equals*

$$\text{encode}(\nu \| m) + F_{s'}(b) + e \quad \text{with} \quad \|e\|_\infty \leq t$$

after t updates. For $t \leq 127$ the noise pad removes e on decryption, and e is independent of the message bits. Distinguishing the updated body from uniform R_p is therefore Ring-LWR under s' . Seeing both the pre- and post-update bodies does not help beyond evaluating F_Δ , which the server can already compute from the public token Δ and the public a_b .

Lemma 3 (IND-DD-UP). *Any PPT adversary $\mathcal{A}$ against IND-DD-UP of our USS satisfies*

$$\text{Adv}_{\mathcal{A}}^{\text{IND-DD-UP}} \leq \text{Adv}_{\mathcal{A}}^{\text{RO-guess}} + \text{Adv}_{\mathcal{B}_1}^{\text{RISE-ENC}} + \text{Adv}_{\mathcal{B}_2}^{\text{RLWR}} + \frac{n}{p},$$

where the first term is the probability that $\mathcal{A}$ queries H on the challenge nonce before it is revealed, $\mathcal{B}_1$ is a RISE IND-ENC adversary under SXDH, and $\mathcal{B}_2$ is a decisional Ring-LWR adversary in the ROM.

Proof. We follow the hybrid outline of [12, Thm. 3]. Let i^* be the challenge slot and (m_0^*, m_1^*) the challenge plaintexts of equal length. Write H_b for hybrids parameterized by the hidden bit $b \in \{0, 1\}$.

Game H_0 (real). The challenger runs ParGen/KeyGen, stores the challenge vector with $m_{i^*} = m_b^*$, and answers FileUp/KeyUp/Rev honestly. $\mathcal{A}$ sees the full repository (RISE openings, Ring-LWR bodies, tweaks) and the stub C .

Game H_1 (hide COM). Identical to H_0 , except the challenger samples $h_{i^*} \leftarrow \mathbb{F}_p$ uniformly instead of $h_{i^*} = \text{COM}(m_b^*; \nu_{i^*})$, and programs the random oracle so that $H(m_b^* \| \nu_{i^*}) = h_{i^*}$ only if $\mathcal{A}$ never queries that point. If $\mathcal{A}$ queries H on $(m_b^* \| \nu_{i^*})$ before ν_{i^*} is opened by an honest Rev, the reduction aborts; otherwise H_0 and H_1 are identical. Since ν_{i^*} has 256 bits of entropy, this abort probability is at most $\text{Adv}_{\mathcal{A}}^{\text{RO-guess}}$ (negligible under standard query bounds).

Game H_2 (hide RISE openings). Replace every RISE ciphertext E_j that opens a KHVC proof depending on h_{i^*} —in particular E_{i^*} and, after FileUp/KeyUp, every opening that has absorbed a challenge-dependent delta—by $\text{RISE.Enc}_k(U_j)$ for independent uniform $U_j \in \mathbb{G}_1$. A standard hybrid over the finitely many such openings reduces to RISE IND-ENC in $\mathbb{G}_1$, which holds under SXDH [4]. Call the resulting adversary $\mathcal{B}_1$.

Game H_3 (hide the body). Replace the Ring-LWR body of slot i^* by independent uniform elements of R_p (same length). By the DEM hop of Remark 3, this is decisional Ring-LWR for the current epoch secret (and, after KeyUp, for the updated secret). Call the resulting adversary $\mathcal{B}_2$.

Game H_4 (hide the stub). $\mathcal{A}$ still sees the stub C and the RISE openings of every slot $j \neq i^*$ (which, after H_2 , encrypt values independent of h_{i^*}). By Theorem 2, the two worlds that differ only in the challenge slot message (and hence only in h_{i^*}) are statistically identical except with probability at most n/p over setup ($\tau \in \Omega$). Thus H_4 is independent of b .

The Type-3 pairing check

$$e(C - [h]_1, [1]_2) = e(\Lambda, [\tau]_2 - [\omega]_2)$$

uses only public $\mathbb{G}_2$ SRS elements $[1]_2, [\tau]_2$; nothing is extracted from $\mathbb{G}_2$ beyond what SXDH already assumes. Summing the hops yields the claimed bound. In H_4 the adversary’s view is independent of b , so $\text{Adv}_{\mathcal{A}}^{\text{IND-DD-UP}}$ is at most the sum of the hop advantages. $\square$

Lemma 4 (IND-FileUp-CPA). *Any PPT adversary $\mathcal{A}$ against IND-FileUp-CPA satisfies*

$$\text{Adv}_{\mathcal{A}}^{\text{IND-FileUp-CPA}} \leq \text{Adv}_{\mathcal{A}}^{\text{RO-guess}} + \text{Adv}_{\mathcal{B}_1}^{\text{RISE-ENC}} + \text{Adv}_{\mathcal{B}_2}^{\text{RLWR}} + \frac{n}{p}.$$

Proof. We follow [12, Thm. 4]. The adversary first obtains an honestly stored repository, then issues a challenge FileUp at slot i^* for two equal-length payloads (m'_0, m'_1) . The FileUp delta is one-hot: only coordinate i^* is nonzero.

Game H_0 (real). The challenger runs an honest FileUp for m'_b , producing a fresh nonce ν' , commitment $h' = \text{COM}(m'_b; \nu')$, one-hot openings $\{\Lambda_{\delta,j}\}_j$ by Lemma 2, RISE encryptions of those openings, a fresh tweak, and a fresh Ring-LWR body under the current DEM secret s .

Game H_1 (hide COM). Replace h' by a uniform $\mathbb{F}_p$ element, programming $H(m'_b \| \nu') = h'$ unless $\mathcal{A}$ queried that point. The hop is identical to H_0 unless the RO-guess event of Lemma 3 occurs.

Game H_2 (hide openings and body). Replace every $\text{RISE.Enc}_k(\Lambda_{\delta,j})$ by an encryption of a uniform $\mathbb{G}_1$ element (RISE IND-ENC / SXDH; hybrid over n openings), and replace the fresh body by uniform R_p elements. The new tweak ensures that every $a_b = H(\text{tweak}' \| b)$ is a fresh oracle string, so the body hop is Ring-LWR under the current s (Remark 3). The one-hot closed form of Lemma 2 produces the same opening group elements as a dense all-open; the reduction does not depend on which algorithm computes them.

Game H_3 (hide the updated stub). The updated stub is $C' = C \oplus C_\delta$ with C_δ committing to a one-hot vector whose only nonzero entry is the (already uniform) $h' - h_{i^*}$. Apply Theorem 2 to C' and to the openings of all slots except i^* : the view is statistically independent of b up to an n/p setup failure.

The same Type-3 pairing check as in Lemma 3 replaces any symmetric-group extraction step of [12]. $\square$

Lemma 5 (IND-REENC-CPA). *Any PPT adversary $\mathcal{A}$ against IND-REENC-CPA that triggers at most $t \leq 127$ body updates satisfies*

$$\text{Adv}_{\mathcal{A}}^{\text{IND-REENC-CPA}} \leq \text{Adv}_{\mathcal{B}_1}^{\text{RISE-UPD}} + \text{Adv}_{\mathcal{B}_2}^{\text{RLWR}} + \frac{n}{p},$$

where $\mathcal{B}_1$ attacks RISE rand-IND-UPD in $\mathbb{G}_1$ and $\mathcal{B}_2$ attacks decisional Ring-LWR for the post-update DEM secret.

Proof. We follow [12, Thm. 5], separating header/stub unlinkability from body unlinkability.

Headers and stub. An honest KeyUp publishes $(\Delta_R = k'/k, Y' = k'g_1, \rho_0)$ and the server (i) applies RISE update with token Δ_R to every opening ciphertext, then (ii) adds $\text{RISE.Enc}_{k'}(\rho_0 W_j)$ to opening j and $[\rho_0 Z(\tau)]_1$ to the stub. Step (i) is exactly the RISE rand-IND-UPD game in $\mathbb{G}_1$ (SXDH) [4]: after the update, each opening ciphertext is distributed as a fresh encryption under k' of the same $\mathbb{G}_1$ message. Step (ii) is a public KHVC rerandomization: adding $\rho_0 Z$ does not change any evaluation on Ω , and by Theorem 2 (applied to a zero-message vector with blinding ρ_0) the updated stub together with the updated openings is statistically independent of the pre-KeyUp blinding, up to an n/p setup failure. A hybrid that first replaces RISE updates by fresh encryptions under k' , then replaces the stub/opening rerandomization by an independent fresh KHVC commitment to the same hash vector, is therefore indistinguishable from the real KeyUp header except for $\text{Adv}_{\mathcal{B}_1}^{\text{RISE-UPD}} + n/p$.

Bodies. The DEM token is $\Delta = s' - s \in R_q$. The server replaces every body block c by $c + F_\Delta(b)$. As in Remark 3, for $t \leq 127$ the resulting body equals a fresh encoding under s' plus a message-independent rounding error of infinity norm at most t . Replacing every updated body by uniform R_p elements is decisional Ring-LWR under s' . The public Δ does not help: it is uniform given s' , and F_Δ is already computable by the server. This hop is $\text{Adv}_{\mathcal{B}_2}^{\text{RLWR}}$.

Challenge bit. After both hops, the post-KeyUp repository (headers, stub, and bodies) is distributed as a fresh Store under (k', s') of the same plaintexts (up to the statistical n/p loss on the stub), independent of any pre-KeyUp challenge bit that IND-REENC asks to hide. Summing yields the bound. $\square$

Lemma 6 (OF-PTXT under key leakage). *OF-PTXT holds by KHVC binding and COM binding in the ROM, even if the current RISE key, the DEM secret, and every KeyUp nonce ρ_0 are leaked. Updating the body is not what the integrity argument uses.*

Proof. The adversary of [12, Thm. 6] may hold the RISE key, the DEM secret, and every KeyUp nonce ρ_0 , and may drive FileUp and KeyUp. A win is a repository that verifies at slot i for a plaintext that is not the latest honest file at i .

After an honest FileUp the stub is $C' = C \oplus C_\delta$ and the honest opening at i verifies for h'_i . A stale plaintext whose commitment is $h_i \neq h'_i$, together with an opening Λ^* that verifies for C' , is two accepting openings of one slot of C' to two messages. That is a KHVC binding break, and the reduction in the binding theorem turns it into q -SDH. If the stale bytes hash to h'_i , the same transcript is a random-oracle collision for COM. The reduction does not need the RISE key.

KeyUp adds $[\rho_0 Z(\tau)]_1$ to the stub and $\rho_0 W_j$ to each opening. Both are multiples of Z , so every evaluation on Ω is unchanged. Reusing a pre-FileUp opening, which verifies the old stub for h_i , does not verify the updated stub for h_i : the updated stub opens to h'_i . A second opening to h_i is again binding. Knowledge of ρ_δ lets the adversary recompute C_δ . It does not let the adversary replace the stub the client stores. $\square$

6 Implementation

A single Rust crate (`uss-khvc`) on arkworks 0.5 (BLS12-381), `aes-gcm` (AES baselines only), and `curve25519-dalek` 4.1. File bodies in `uss.rs` are the Ring-LWR DEM with cached a_b NTTs; KeyUp adds F_Δ to every block without rebuilding a_b . FileUp openings use `open_one_hot`; dense FK20 remains for microbenchmarks of $k = n$. The USS timings in Table 8 are one-shot measurements from `experiments/results/uss.csv` on this build. KHVC uses `Radix2EvaluationDomain` IFFTs and an explicit $\mathbb{G}_1$ Cooley–Tukey FFT for FK20. Catalano–Fiore HVC stores $h_i \in \mathbb{G}_1$, $\hat{h}_i \in \mathbb{G}_2$, and off-diagonal $h_{i,j} \in \mathbb{G}_1$ (Type-3 verification $e(C - mh_i, \hat{h}_i) = e(\Lambda_i, g_2)$). UE baselines live in the same process so Enc/Dec/Upd numbers are comparable. The crate now selects the naïve all-open for $n < 1024$ and FK20 otherwise. Table 1 and the USS rows are one-shot timings from a CSV that predates that switch. The switch does not change the $n \geq 1024$ cells, and the small- n USS rows were not remeasured under it.

We also looked for public UE code. SHINE has a Python/C prototype [19]; ReCrypt has an abandoned Rust prototype [20]; nested AES and RLWE KH-PRF have C code [21]. IronCore `recrypt-rs` is proxy re-encryption, not UE. RISE has no public implementation; we wrote it. Primary tables use the same Rust stack; we did not mix MIRACL C timings into the main table.

7 Evaluation

Machine. Apple M5 Pro, 24 GB RAM, 15 threads, macOS 26.6.2, rustc 1.97.1, release with LTO. HVC and USS cells are one-shot timings from `experiments/results/all.csv`. Tables 2–3 are medians from `fairbench` (`experiments/results/fair_ue.csv`): a discarded warm-up, then 21 repeats at header size and 4 KB, 5 at 64 KB, 3 at 1 MB algebraic, and 7 at 1 MB AES/KH-PRF. Table 4 is the same median accounting from `rlwrbench` (`experiments/results/rlwr_ue.csv`): warm-up discarded, then 21 repeats at 4 KB, 7 at 64 KB, and 5 at 1 MB, single-threaded. We do not time RISE or SHINE0 at 100 MB (1 MB RISE Enc is already 2.6 s). A 100 MB AES/Nested figure is taken from the historical `bench` suite; ReCrypt at 100 MB is $100\times$ the 1 MB median and is labeled extrapolation.

7.1 HVC: linear vs quadratic CRS

Table 1: KHVC vs Catalano–Fiore HVC. Times in milliseconds, one-shot. The CRS column is the verification counter (`crs_bytes`): it excludes $\{W_j\}$ and includes one redundant 48-byte $[Z(\tau)]_1$. CF-HVC is omitted for $n \geq 4096$.

n	scheme	CRS	setup	Com	Open+Ver	all-open
64	KHVC	3.3 KB	17	0.86	2.20	62 (FK20) / 36 (naive)
64	CF-HVC	204 KB	298	3.86	5.22	259
256	KHVC	12.3 KB	79	1.03	2.24	326 / 216
256	CF-HVC	3.05 MB	4475	15.5	16.8	3995
1024	KHVC	48.3 KB	379	2.36	3.45	1624 / 1694
1024	CF-HVC	50.5 MB	70249	61.8	63.0	63131
4096	KHVC	192 KB	1734	6.39	7.07	7707
16384	KHVC	768 KB	8200	17.9	18.0	36495

The verification-SRS ratio at $n = 1024$ is $50.5 \text{ MB} / 48.3 \text{ KB} \approx 1070$. Adding the KeyUp table gives operational parameters of 96.3 KB, and $50,528,400 / (49,440 + 49,152) \approx 512$. A Catalano–Fiore CRS at $n = 2^{14}$ would be $(16384/1024)^2 \cdot 50.5 = 12.8 \text{ GB}$;¹ the KHVC verification SRS is 768 KB and the operational parameters are about 1.5 MB. Table 1 shows FK20 losing

¹ $(2^{14}/2^{10})^2 = 256$ and $256 \times 50.5 \text{ MB} = 12.8 \text{ GB}$.

to the naïve all-open for $n < 1024$. Use the naïve path there. The published CSV predates the switch in the crate; it does not change these $n \geq 1024$ cells. KeyUp with precomputed Z -openings is 63 ms at $n = 1024$ and 1.10 s at $n = 2^{14}$, versus a full FK20 all-open of 1.62 s and 36.5 s.

7.2 Payload UE

A 48 B RISE Enc and a 4 KB AES Enc must not share a millisecond column: on the same 4 KB plaintext the costs are 2428 ns/B versus 4.21 ns/B (577 $\times$). We therefore split header latency from same-size throughput. The ReCrypt-style row is a classical elliptic-curve KH-PRF on Ristretto (DDH), not the RLWE construction of [7].

Table 2: Header UE (one group element). Median milliseconds after warm-up. Token is compact and independent of file size. AES has no server token.

scheme	$ m $	Enc	Upd	Dec	$ token $	$ ct / m $
RISE ($\mathbb{G}_1$)	48 B	0.24	0.36	0.12	80 B	2
SHINE0 (Ristretto)	32 B	0.027	0.027	0.038	32 B	1

Table 3: Same $|m|$ payload UE. Enc/Upd/Dec in MB/s. Algebraic schemes are block-wise (RISE: 48 B $\mathbb{G}_1$; SHINE0 and ReCrypt-style KH-PRF: 32 B Ristretto). Median after warm-up. KHVC-USS uses the Ring-LWR DEM of Table 4 for bodies and Table 2 for openings; the USS KeyUp comparison is Table 7.

scheme	$ m $	Enc	Upd	Dec	Enc ns/B	$ ct / m $
RISE ($\mathbb{G}_1$)	4 KB	0.41	0.27	0.76	2428	2.02
SHINE0	4 KB	1.99	1.97	1.41	503	1.00
ReCrypt-KHPRF (EC)	4 KB	1.28	1.28	7.25	782	2.00
AES-256-GCM	4 KB	237	—*	239	4.21	1.01
Nested AES-GCM	4 KB	238	231	118 [†]	4.20	1.01
RISE ($\mathbb{G}_1$)	1 MB	0.40	0.26	0.73	2490	2.00
SHINE0	1 MB	1.93	1.92	1.37	519	1.00
ReCrypt-KHPRF (EC)	1 MB	1.29	1.30	7.52	776	2.00
AES-256-GCM	1 MB	241	—*	241	4.16	1.00
Nested AES-GCM	1 MB	247	246	125 [†]	4.05	1.00

* no compact server token; a client re-encrypt is 238 MB/s at 4 KB and 240 MB/s at 1 MB. [†] decrypt after one nested update (two AES layers).

Headers are the USS workload: sub-millisecond Enc/Upd/Dec and an $O(1)$ token (Table 2). Bulk is not. On the same 1 MB plaintext, AES-GCM encrypts at 241 MB/s (4.16 ns/B). SHINE0 is 1.93 MB/s (125 $\times$ slower), the ReCrypt-style KH-PRF is 1.29 MB/s (187 $\times$), and RISE is 0.40 MB/s (600 $\times$). The 4 KB column is the same story (237 vs 1.99 vs 0.41 MB/s). AES Upd is not a token update: the server cannot refresh the body without the key. Nested AES matches Enc throughput but Dec is linear in the number of epochs. At the 128-update bound that linear walk is 128 AES decryptions, so nested AES is not a stable DEM either. A historical one-shot 100 MB AES Enc is 0.45 s; the corresponding ReCrypt figure is 81 s if the 1 MB median is scaled by 100 (extrapolation, not a timed run).

7.3 Ring-LWR DEM, sized for 128 updates

The DEM is the random-oracle Ring-LWR scheme recalled in [7, §6]: for each block index i , $F_s(i) = \lfloor (p/q)(a_i \cdot s) \rfloor$ in R_p with public $a_i = H(i)$, and the token is $\Delta = s' - s \in R_q$. It is one negacyclic product per block, not the standard-model tree PRF that multiplies once per input bit.

Parameters are a performance point, not a lattice-estimator claim: $N = 1024$, $q = 998244353$, $p = 2^{16}$, eight message bits and eight noise bits per coefficient. Each update adds a rounding error of at most 1 per coefficient, so eight noise bits guarantee 127 updates. That is the 128-update bound up to a single rotation. Past that bound the pad overflows in the worst case, and a longer archive or a monthly rotation needs a wider pad or a client re-encryption. Ciphertext expansion is 2. The token is N elements of $\mathbb{Z}_q$, hence 4096 bytes, independent of $|m|$ and of the number of files. If the NTT of every a_i is cached, that table is four times the plaintext (4 MB for a 1 MB file) and is not counted in Enc/Upd/Dec; building it for 1 MB took 35 ms.

Table 4: Ring-LWR KH-PRF DEM. Median MB/s after warm-up, single-threaded reference NTT. Public a_i excluded.

$ m $	Enc	Upd	Dec	ns/B	$ ct / m $
4 KB	33.4	33.8	34.4	30.0	2
64 KB	33.0	32.8	33.7	30.3	2
1 MB	32.6	32.6	32.9	30.7	2

On 64 blocks the largest absolute rounding error was 42 after 64 updates and 78 after 128 updates, both inside the pad ($|e| < 128$). On 2 blocks it was 144 after 255 updates, which the pad does not survive. Four 1 GB bodies at the 1 MB update median of Table 4 would be about 126 s of pure DEM work when a_b is cached. The end-to-end USS KeyUp of Table 8 stores those NTTs with each file and measures 254 s for the same four files, with a 4240-byte token (previously 404 s when a_b was rebuilt from the tweak on every block). The pure DEM estimate of 126 s assumes the 1 MB Upd median and no memory pressure; at 4×1 GB the a_b cache is about $8 \times$ the plaintext and the measured wall-clock sits between that estimate and the old rebuild path. The server rewrites a $2 \times$ ciphertext locally. This is the payload rotation the 128-update policy actually costs. It is still about an order of magnitude faster than SHINE0 or RISE on the same bytes, and AES-GCM cannot update.

Concrete cost of these Ring-LWR parameters. We estimated the DEM as unstructured LWE with the lattice-estimator [10], commit 53da598: dimension 1024, modulus $q = 998244353$, secret uniform in $\mathbb{Z}_q$, error uniform on $[-7616, 7616]$ (the centred rounding width $q/(2p)$, standard deviation ≈ 4397), and $m = 2^{20}$ samples, which is one ring element per block of a 1 GB file. Raising m to 2^{22} does not change the figure. The cheapest finite attack the estimator returned is BDD at $\approx 2^{319}$ operations; uSVP is $\approx 2^{322}$ and the dual attack is $\approx 2^{334}$. Coded-BKW did not return a finite cost on this instance. The model does not use the cyclotomic ring, so this is a generic-lattice cost of the same dimension and noise, not a reduction from ideal-SVP. It is the reason these parameters are a security point and not only a timing point: every attack the estimator finished is far above 2^{128} .

7.4 End-to-end USS

Table 3 is not a USS comparison: it times encryption engines. USS is the 2023 protocol with a new HVC backend and a measured large-file path. The two comparisons that show the construction’s advantage are Table 5 (against the only published USS) and Table 7 (against rotating the same repository without a stub).

The durable gain against the 2023 backend is the CRS: at $n = 1024$ the operational parameters are $512 \times$ smaller, and at $n = 2^{14}$ Catalano–Fiore is 12.8 GB against about 1.5 MB. One-hot FileUp openings cut the single-slot KHVC cost from 1.62 s to 194 ms at $n = 1024$ (about $3 \times$ Catalano–Fiore’s estimated 62 ms, down from a $26 \times$ gap when FK20 was used for $k = 1$). Dense FK20 is still about 1.67 s independent of $k \geq 8$, while the dense CF estimate scales as about $61.7k$ ms, so the crossover remains near $k^* \approx 27$. The dense case $k = n$ is where KHVC is about

Table 5: The ASIACRYPT 2023 USS protocol with Catalano–Fiore HVC versus this paper. HVC cells are the same-stack one-shot measurements of Table 1. Verification CRS excludes $\{W_j\}$.

	2023 CF-USS	KHVC-USS
Verification CRS, $n = 1024$	50.5 MB	48.3 KB (1070 $\times$)
Operational CRS, $n = 1024$	50.5 MB	96.3 KB (512 $\times$)
ParGen, $n = 1024$	70.2 s	0.38 s (185 $\times$)
CRS, $n = 2^{14}$	12.8 GB [§]	768 KB / ≈ 1.5 MB
KeyUp, 4×1 GB	— (no public impl.)	254 s / 4240 B

[§] $(16384/1024)^2 \cdot 50.5 \text{ MB} = 12.8 \text{ GB}$. The 512 $\times$ ratio is $50,528,400/(49,440 + 49,152)$. FileUp cost depends on k ; see Table 6.

Table 6: Server opening work for a FileUp that touches k slots, $n = 1024$. KHVC $k = 1$ uses the one-hot closed form (Lemma 2); denser k use FK20. CF cells with $k < n$ are $k \cdot 63.13 \text{ s}/1024$, a linear estimate from the measured dense all-open, not a sparse CF implementation.

k	CF-HVC	KHVC	source
1	$\approx 62 \text{ ms}$	194 ms	CF estimated, KHVC one-hot
8	$\approx 0.49 \text{ s}$	1.67 s	CF estimated, KHVC FK20
64	$\approx 3.9 \text{ s}$	1.69 s	CF estimated, KHVC FK20
n	63.1 s	1.67 s	both measured (38 $\times$ here)

Table 7: Key rotation of the *same* repository. KHVC-USS KeyUp is measured with cached a_b NTTs, including the Ring-LWR body scan. Baselines re-key every file body. Algebraic-file rows have no stub and no OF-PTXT.

scheme	$n \times m $	time	sent	OF-PTXT
KHVC-USS	$4 \times 1 \text{ GB}$	254 s	4240 B	yes
AES client re-enc	$4 \times 1 \text{ GB}$	17.9 s [‡]	4.0 GB	no
Nested AES Upd	$4 \times 1 \text{ GB}$	17.5 s [‡]	4.0 GB	no
ReCrypt-KHPRF (EC)	$4 \times 1 \text{ GB}$	55 min [‡]	8.0 GB	no
SHINE0 on bodies	$4 \times 1 \text{ GB}$	37 min [‡]	4.0 GB	no
RISE on bodies	$4 \times 1 \text{ GB}$	4.6 h [‡]	8.0 GB	no
KHVC-USS	$8 \times 100 \text{ MB}$	58.9 s	4240 B	yes
AES client re-enc	$8 \times 100 \text{ MB}$	3.5 s [‡]	800 MB	no
KHVC-USS	$64 \times 1 \text{ MB}$	2.22 s	4240 B	yes
AES client re-enc	$64 \times 1 \text{ MB}$	0.28 s	64 MB	no

[‡] Not a timed run. $T = T_{1\text{MB}} \cdot n \cdot |m|/1 \text{ MB}$ from **fairbench** medians (AES: client re-encrypt; others: server Upd). Block-wise algebraic throughput is essentially flat in $|m|$; a 1 GB working set may still see cache and memory-bandwidth effects not present at 1 MB.

$38\times$ faster. That is why a CF-USS 1 GB experiment is missing: the 2023 CRS does not scale to the n where USS is interesting.

Against payload-only rotation of the same four 1 GB files, KHVC-USS KeyUp is measured at 254 s and 4240 B, including the Ring-LWR body scan with cached a_b . AES re-encrypt is 17.9 s and 4 GB on the wire; SHINE0 or RISE on the file bytes is 37 minutes to 4.6 hours. Those algebraic rows are extrapolations, not timed runs. The USS row is slower than client AES re-encrypt on this machine, but the token is $10^6\times$ smaller than rewriting the bodies, the server does the work, and OF-PTXT is kept.

Table 8: KHVC-USS microbenchmarks (Ring-LWR bodies, cached a_b). FileUp wall-clock includes decrypting file i , encrypting the replacement, and the one-hot open of the delta; KeyUp includes rewriting every body.

n	$ m_i $	ParGen	Store	Rev	FileUp	KeyUp
8	4 KB	3.9 ms	8.0 ms	1.9 ms	4.8 ms	7.5 ms
32	4 KB	7.6 ms	34.5 ms	2.0 ms	19.5 ms	26.3 ms
64	1 MB	16.5 ms	6.08 s	38 ms	0.15 s	2.22 s
8	100 MB	1.6 ms	80.9 s	7.82 s	14.4 s	58.9 s
4	1 GB	1.0 ms	425 s	79.7 s	192 s	254 s

Store, Rev, FileUp, and KeyUp all scale with total payload bytes under the Ring-LWR DEM. KeyUp of four 1 GB files is 254 s: the DEM scan dominates, and the token is 4240 bytes independent of n and of $|m|$. Caching a_b removes the previous rebuild tax (404 s end-to-end with on-the-fly NTT). The extra integrity work for a one-file FileUp is the one-hot open of Lemma 2, not a scan of other plaintexts.

Honest costs. Our FK20 uses projective $\mathbb{G}_1$ arithmetic without a dedicated Pippenger MSM for the Toeplitz step; at $n < 1024$ it loses to the naïve loop in Table 1. The crate now switches there. The CSV behind that table is older than the switch. A production port (blst + FK20 as in Ethereum *c-kzg-4844*) would improve the $n = 2^{14}$ all-open. We did not implement a full CF-HVC USS repository at 1 GB: Table 5 compares the HVC backend that 2023 actually uses, which is the bottleneck.

8 Related work

Vector commitments [2, 13, 14] and polynomial commitments [3] are standard. Fleischhacker–Hall–Andersen–Simkin [16] use hidden KZG as a vector commitment and as a handle for extractable witness encryption. Their interface is a (hiding) polynomial commitment / VC, not an HVC: they do not provide the USS *aux-independent-of- n* contract, **ComHom/OpenHom**, or the position-hiding game with openings of the other slots as group elements. Using a polynomial commitment as a VC is common in proof systems, including Lasso [17]. Those uses likewise omit homomorphism plus short *aux*, which is what FileUp/KeyUp need.

Transparent FRI commitments [18] avoid a pairing SRS. A FRI opening is not a $\mathbb{G}_1$ element, so it does not plug into the RISE homomorphism used here. That is why this USS instantiation uses KZG.

UE: BLMR, RISE [4], ReCrypt [5], SHINE [6], nested AES and an RLWE KH-PRF [7]. That paper already notes that a Ring-LWR almost key-homomorphic PRF exists and moves to RLWE to widen the noise budget; with a 128-update cap the Ring-LWR pad is enough, and Section 7 measures it. USS [12] is the integrity-aware combination and already states that the DEM may be any IND-CPA UE; this paper is its first linear-SRS instantiation with a measured large-file DEM. Cryptoperiods are NIST’s [11], not a new UE definition.

9 Conclusion

KHVC removes the quadratic CRS that blocked the 2023 USS at realistic n (operational parameters $512\times$ smaller at $n = 1024$). Position hiding is statistical, up to an n/p setup event. Body confidentiality and body update are Ring-LWR in the random oracle: IND-CPA for a fresh body and for a body after at most 127 updates. OF-PTXT does not need the DEM to be authenticated, and it holds if the DEM secret leaks. One-hot FileUp openings bring a single-slot update to 194 ms at $n = 1024$ (about $3\times$ the Catalano–Fiore sparse estimate). On four 1 GB files end-to-end KeyUp is 254 s with a 4240-byte token and cached a_b NTTs. Generic-lattice estimation of this Ring-LWR point puts the cheapest finished attack near 2^{319} , above the 2^{128} line, without using the ring.

References

- [1] D. Boneh, X. Boyen. Short signatures without random oracles. EUROCRYPT 2004.
- [2] D. Catalano, D. Fiore. Vector commitments and their applications. PKC 2013.
- [3] A. Kate, G.M. Zaverucha, I. Goldberg. Constant-size commitments to polynomials and their applications. ASIACRYPT 2010.
- [4] A. Lehmann, B. Tackmann. Updatable encryption with post-compromise security. EUROCRYPT 2018.
- [5] A. Everspaugh, K. Paterson, T. Ristenpart, S. Scott. Key rotation for authenticated encryption. CRYPTO 2017. Implementation: <https://github.com/samscott89/ReCrypt>.
- [6] C. Boyd, G.T. Davies, K. Gjøsteen, Y. Jiang. Fast and secure updatable encryption. CRYPTO 2020, ePrint 2019/1457. Implementation: <https://github.com/YaoJGalteland/SHINE>.
- [7] D. Boneh, S. Eskandarian, S. Kim, M. Fisch. Improving speed and security in updatable encryption schemes. ASIACRYPT 2020, ePrint 2020/222. Implementation: https://github.com/moshih/UpdateableEncryption_Code.
- [8] D. Boneh, K. Lewi, H. Montgomery, A. Raghunathan. Key homomorphic PRFs and their applications. CRYPTO 2013.
- [9] A. Banerjee, C. Peikert, A. Rosen. Pseudorandom functions and lattices. EUROCRYPT 2012.
- [10] M.R. Albrecht, R. Player, S. Scott. On the concrete hardness of Learning with Errors. Journal of Mathematical Cryptology 9(3), 2015. Code: <https://github.com/malb/lattice-estimator>.
- [11] National Institute of Standards and Technology. Recommendation for key management: Part 1 — general. NIST Special Publication 800-57 Part 1, Revision 5, 2020. Table 1: symmetric data-encryption keys, originator-usage period < 2 years.
- [12] L. Chen, H. Guo, Y.-N. Li, Q. Tang. Efficient secure storage with version control and key rotation. ASIACRYPT 2023, ePrint 2023/1421.
- [13] M. Campanelli, D. Fiore, N. Greco, D. Kolonelos, L. Nizzardo. Incrementally aggregatable vector commitments. ASIACRYPT 2020.
- [14] D. Boneh, B. Bünz, B. Fisch. Batching techniques for accumulators. CRYPTO 2019.
- [15] D. Feist, D. Khovratovich. Fast amortized KZG proofs. ePrint 2023/033 (FK20/FK22).

- [16] N. Fleischhacker, M. Hall-Andersen, M. Simkin. Extractable witness encryption for KZG commitments. ASIACRYPT 2024.
- [17] S. Setty, J. Thaler, R. Wahby. Unlocking the lookup singularity with Lasso. EUROCRYPT 2024.
- [18] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. ICALP 2018.
- [19] <https://github.com/YaoJGalteland/SHINE>
- [20] <https://github.com/samscott89/ReCrypt>
- [21] https://github.com/moshih/UpdateableEncryption_Code